PROJECT REPORT

DIGITAL SPEEDOMETER FOR BIKE / CAR

Submitted by: Angel Grace Prince

Registered Email ID: angelgraceprince3333@gmail.com

Department: ECE

Government College Of Engineering Kannur

Academic Year 2024–2028

Abstract

A digital speedometer is an important vehicle monitoring system used to determine and display the speed of a vehicle. The purpose of this project is to develop a simple digital speedometer prototype capable of calculating vehicle speed based on wheel rotational speed.

The proposed system uses the rotational speed of a vehicle wheel, represented by revolutions per minute (RPM), together with the wheel diameter to calculate the corresponding vehicle speed in kilometres per hour. For the software prototype, Python Flask is used as the backend framework and HTML, CSS, and JavaScript are used to develop the web-based user interface.

Instead of using an RPM slider, the developed prototype uses RPM control buttons. The buttons simulate wheel rotational input by increasing or decreasing the RPM value. The system calculates the corresponding speed and displays it on a digital dashboard.

The prototype also provides START, STOP, and RESET controls and displays the simulated wheel RPM, wheel diameter, current speed, and trip distance. The system provides a simple method for demonstrating the operation of a digital speedometer in a software environment.

Contents

1 Introduction **6**

1.1 Problem Statement 6

1.2 Need for the Project 7

2 Objectives **8**

3 Literature Review and Existing System **9**

3.1 Existing System 9

3.2 Conventional Speed Measurement 9

3.3 Limitations of a Hardware-Only Prototype 10

3.4 Proposed System 10

3.5 Comparison of Existing and Proposed System 11

4 Methodology **12**

4.1 System Overview 12

4.2 System Flow 13

4.3 System Block Diagram 13

4.4 Software Architecture 14

4.5 Project Structure 14

4.6 Circuit Diagram 15

5 Implementation and Code **16**

5.1 Flask Backend 16

5.2 Complete Python Code 16

5.3 HTML Code 18
5.4 CSS Code 27

6 Output Snapshots **33**
6.1 Initial System State 33
6.2 Low-Speed Operation 34
6.3 Medium-Speed Operation 34
6.4 High-Speed Operation 35
6.5 Flask Server 36

7 Result and Analysis **38**
7.1 Result 38
7.2 Testing 38
7.3 Analysis 39
7.4 Advantages 39
7.5 Limitations 40

8 Conclusion **41**

List of Figures

6.1 Digital Speedometer Initial State 33
6.2 Digital Speedometer at 100 RPM 34
6.3 Digital Speedometer at 200 RPM 35
6.4 Digital Speedometer at 300 RPM 36
6.5 Flask Server Running in VS Code 37

List of Tables

3.1 Comparison of Existing and Proposed Systems 11

4.1 Software Technologies 14

7.1 Speed Calculation Test Results 39

Chapter 1

Introduction

A speedometer is a device used in vehicles to indicate the speed at which the vehicle is travelling. Conventional speedometers may use mechanical or electronic sensing mechanisms to determine vehicle speed. Modern digital speedometers provide speed information in a clear numerical format and can also support additional parameters such as distance travelled.

The project presented in this report is a **Digital Speedometer for Bike/Car**. The system is designed to calculate vehicle speed using wheel rotational information.

The speed of a vehicle can be calculated from the wheel rotational speed when the wheel diameter is known. The circumference of the wheel determines the distance travelled during one complete revolution.

For this project, the software prototype uses RPM as the simulated wheel-rotation input. The RPM is controlled using buttons rather than a slider. This provides a simple and reliable method for demonstrating different vehicle speeds.

The project is implemented using Python Flask, HTML, CSS, and JavaScript. Flask provides the web application framework, while the frontend provides the digital speedometer interface.

1.1 Problem Statement

Traditional vehicle speed measurement systems require appropriate sensors and hardware to detect wheel rotation or vehicle movement. For an academic software prototype, direct hardware implementation may not always be available.

Therefore, the problem addressed in this project is to develop a software-based digital speedometer that can simulate wheel rotational input and calculate the corresponding

vehicle speed.

1.2 Need for the Project

The project is useful for demonstrating the principle of digital speed measurement without requiring a complete physical vehicle setup.

The software prototype provides:

- A digital speed display.
- RPM-based speed calculation.
- Wheel diameter calibration.
- START and STOP control.
- RESET functionality.
- Trip distance display.
- A simple browser-based user interface.

Chapter 2

Objectives

The main objective of the project is to develop a digital speedometer prototype that calculates and displays vehicle speed using wheel RPM and wheel diameter.

The specific objectives are:

1. To develop a digital speedometer using Python Flask.
2. To calculate vehicle speed from wheel RPM.
3. To use wheel diameter as a calibration parameter.
4. To provide a clear digital display of vehicle speed.
5. To simulate wheel rotational input using RPM control buttons.
6. To provide START, STOP, and RESET controls.
7. To display the current wheel RPM.
8. To calculate and display trip distance.
9. To develop a simple web-based interface using HTML, CSS, and JavaScript.
10. To test the speedometer at different simulated RPM values.

Chapter 3

Literature Review and Existing System

3.1 Existing System

Vehicle speedometers are commonly used to indicate the instantaneous speed of a vehicle. In traditional systems, vehicle speed can be obtained using mechanical arrangements, wheel sensors, magnetic sensors, optical sensors, or GPS-based methods.

In an electronic speedometer, a sensor can detect wheel rotation and generate pulses corresponding to wheel movement. The number of rotations within a particular period can be used to determine the rotational speed of the wheel.

If the wheel circumference is known, the rotational speed can be converted into linear vehicle speed.

3.2 Conventional Speed Measurement

A conventional electronic speed measurement system can contain:

- Wheel rotation sensor.
- Signal conditioning circuit.
- Microcontroller.
- Speed calculation algorithm.
- Digital display.

A Hall-effect sensor or another suitable sensor can be used to detect wheel rotation. The resulting signal can be processed by a microcontroller.

3.3 Limitations of a Hardware-Only Prototype

A physical implementation requires:

- Appropriate sensors.
- Vehicle/wheel arrangement.
- Microcontroller hardware.
- Display hardware.
- Power supply.
- Electrical connections.

For an initial software demonstration, these hardware components may not be readily available.

3.4 Proposed System

The proposed software prototype simulates wheel rotational input using RPM control buttons.

The user can increase or decrease the RPM using the buttons. The system then calculates the corresponding vehicle speed using the wheel diameter.

The software architecture consists of:

RPM Input

↓

Speed Calculation

↓

Digital Display

The Flask backend provides the web application while the browser provides the user interface.

3.5 Comparison of Existing and Proposed System

Table 3.1: Comparison of Existing and Proposed Systems

Existing System	**Proposed Software Prototype**
Requires physical sensing mechanism	RPM is simulated using control buttons
Requires vehicle/wheel hardware	No physical vehicle is required for demonstration
Requires display hardware	Browser-based digital display
Speed calculation performed by hardware/software	Speed calculated by the web application
Difficult to modify for demonstration	RPM can be easily changed using buttons

Chapter 4

Methodology

4.1 System Overview

The Digital Speedometer is implemented as a web-based application.

The user interacts with the RPM control buttons through the web browser. The selected RPM value is processed by the JavaScript calculation logic.

The wheel circumference is calculated using:

$$C = \pi D \tag{4.1}$$

where:

- C = wheel circumference in metres
- D = wheel diameter in metres

The distance travelled per minute is:

$$Distance_{min} = RPM \times C \tag{4.2}$$

The speed in kilometres per hour is:

$$Speed = \frac{RPM \times C \times 60}{1000} \tag{4.3}$$

Substituting $C = \pi D$:

$$Speed = \frac{RPM \times \pi \times D \times 60}{1000} \tag{4.4}$$

For this prototype:

$$D = 0.66\ m$$

Therefore:

$$Speed = \frac{RPM \times \pi \times 0.66 \times 60}{1000} \tag{4.5}$$

4.2 System Flow

The working sequence of the system is:

1. Start the Flask application.
2. Open the application in a web browser.
3. The initial RPM is set to zero.
4. Press the **START** button.
5. Increase the simulated RPM using the **+50 RPM** button.
6. The system calculates the corresponding speed.
7. The speed is displayed in km/h.
8. The trip distance is updated while the vehicle is running.
9. The **STOP** button stops the simulated vehicle.
10. The **RESET** button returns the system to its initial state.

4.3 System Block Diagram

The conceptual block diagram of the software prototype is:

RPM Control Buttons

↓

RPM Value

↓

Wheel Circumference Calculation

↓

Speed Calculation

↓

Digital Speed Display

↓

Trip Distance Display

4.4 Software Architecture

The project uses the following technologies:

Table 4.1: Software Technologies

Technology	**Purpose**
Python	Backend programming
Flask	Web application framework
HTML	Web page structure
CSS	User interface design
JavaScript	RPM controls and calculations
VS Code	Development environment
Web Browser	Application testing

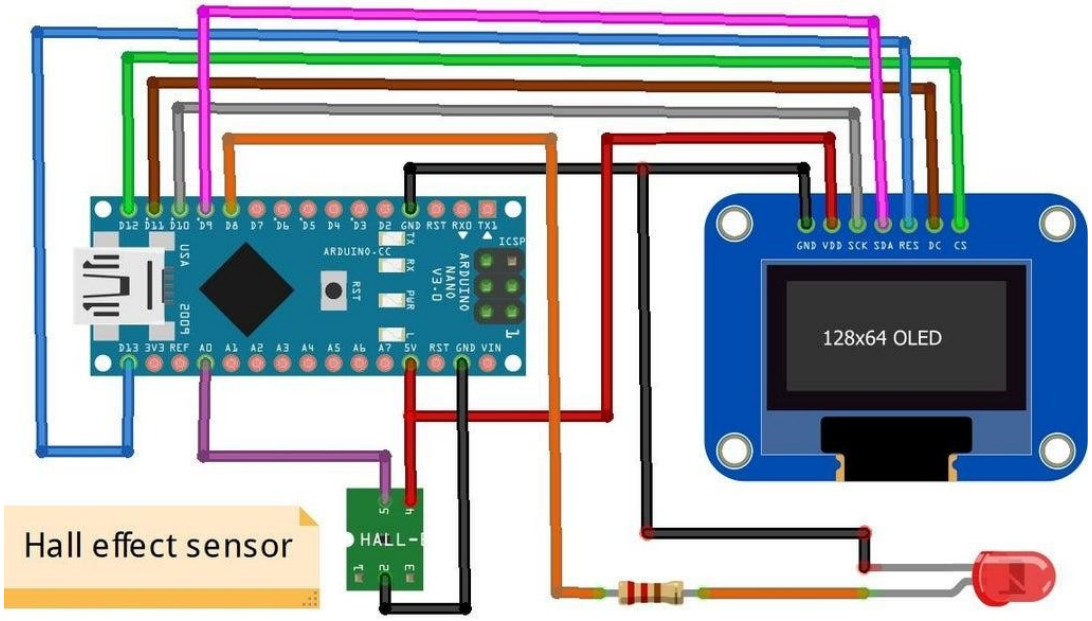
4.5 Project Structure

The project is organized as follows:

```
DigitalSpeedometer/
|
|-- app.py
|-- requirements.txt
|
|-- templates/
```

```
|   `-- index.html
|
`-- static/
    `-- style.css
```

4.6 Circuit Diagram



Chapter 5

Implementation and Code

5.1 Flask Backend

The backend is implemented using Python Flask.

The Flask application provides the main web page and a calculation endpoint.

5.2 Complete Python Code

The following code is the Flask backend used in the project.

```
1
2  from flask import Flask, render_template, jsonify, request
3  import math
4
5  app = Flask(__name__)
6
7  # Default wheel diameter
8  DEFAULT_WHEEL_DIAMETER = 0.66
9
10
11 @app.route("/")
12 def home():
13
14     return render_template(
15         "index.html",
16         wheel_diameter=DEFAULT_WHEEL_DIAMETER
17     )
18
19
20 @app.route("/calculate", methods=["POST"])
```

```
def calculate():

    data = request.get_json(silent=True) or {}

    try:
        rpm = float(data.get("rpm", 0))
    except (TypeError, ValueError):
        rpm = 0.0

    try:
        wheel_diameter = float(
            data.get(
                "wheel_diameter",
                DEFAULT_WHEEL_DIAMETER
            )
        )
    except (TypeError, ValueError):
        wheel_diameter = DEFAULT_WHEEL_DIAMETER

    rpm = max(0.0, min(rpm, 1000.0))

    if wheel_diameter <= 0:
        wheel_diameter = DEFAULT_WHEEL_DIAMETER

    circumference = math.pi * wheel_diameter

    metres_per_minute = rpm * circumference

    speed_kmh = (
        metres_per_minute * 60
    ) / 1000

    return jsonify({
        "rpm": round(rpm, 2),
        "wheel_diameter": round(
            wheel_diameter, 2
        ),
        "speed_kmh": round(
            speed_kmh, 2
        ),
        "circumference": round(
            circumference, 3
        )
    })


if __name__ == "__main__":
```

```
68
69      app.run(
70          host="127.0.0.1",
71          port=5001,
72          debug=True
73      )
```

Listing 5.1: Complete Flask Backend – app.py

5.3 HTML Code

The main interface is implemented using HTML.

```
1
2  <!DOCTYPE html>
3  <html lang="en">
4
5  <head>
6
7      <meta charset="UTF-8">
8
9      <meta name="viewport"
10         content="width=device-width,
11                  initial-scale=1.0">
12
13     <title>Digital Speedometer</title>
14
15     <link rel="stylesheet"
16         href="{{ url_for('static',
17         filename='style.css') }}">
18
19 </head>
20
21 <body>
22
23 <div class="container">
24
25     <header>
26
27         <h1>DIGITAL SPEEDOMETER</h1>
28
29         <p>Bike / Car Speed Monitoring System</p>
30
31     </header>
32
33
```

```
34      <section class="speed-card">
35
36          <p class="display-title">
37              CURRENT SPEED
38          </p>
39
40          <div class="speed-display">
41
42              <span id="speedValue">0.00</span>
43
44          </div>
45
46          <div class="unit">
47              km/h
48          </div>
49
50          <div id="statusValue"
51               class="status stopped">
52
53              VEHICLE STOPPED
54
55          </div>
56
57      </section>
58
59
60      <section class="control-card">
61
62          <h2>Wheel Rotation Simulation</h2>
63
64          <p class="description">
65              Use the buttons below to simulate
66              wheel RPM.
67          </p>
68
69          <div class="rpm-display">
70
71              <span id="rpmValue">0</span>
72
73              <span> RPM</span>
74
75          </div>
76
77
78          <div class="button-row">
79
80              <button id="decreaseBtn"
```

```
                    class="control-button">

                - 50 RPM

            </button>

            <button id="increaseBtn"
                    class="control-button">

                + 50 RPM

            </button>

        </div>


        <div class="button-row">

            <button id="startBtn"
                    class="start-button">

                START

            </button>

            <button id="stopBtn"
                    class="stop-button">

                STOP

            </button>

            <button id="resetBtn"
                    class="reset-button">

                RESET

            </button>

        </div>

    </section>


    <section class="info-card">

        <h2>Vehicle Information</h2>
```

```
128
129         <div class="info-grid">
130
131             <div class="info-item">
132
133                 <span class="info-label">
134                     Wheel Diameter
135                 </span>
136
137                 <span class="info-value">
138                     <span id="diameterValue">
139                         {{ wheel_diameter }}
140                     </span>
141                     m
142                 </span>
143
144             </div>
145
146
147             <div class="info-item">
148
149                 <span class="info-label">
150                     Wheel RPM
151                 </span>
152
153                 <span class="info-value">
154                     <span id="rpmInfo">
155                         0
156                     </span>
157                     RPM
158                 </span>
159
160             </div>
161
162
163             <div class="info-item">
164
165                 <span class="info-label">
166                     Current Speed
167                 </span>
168
169                 <span class="info-value">
170                     <span id="speedInfo">
171                         0.00
172                     </span>
173                     km/h
174                 </span>
```

```
175
176             </div>
177
178
179             <div class="info-item">
180
181                 <span class="info-label">
182                     Trip Distance
183                 </span>
184
185                 <span class="info-value">
186                     <span id="tripDistance">
187                         0.000
188                     </span>
189                     km
190                 </span>
191
192             </div>
193
194         </div>
195
196     </section>
197
198
199     <section class="formula-card">
200
201         <h2>Speed Calculation</h2>
202
203         <p>
204             Speed =
205             RPM x pi x Wheel Diameter x 60 / 1000
206         </p>
207
208     </section>
209
210
211     <footer>
212
213         <p>
214             Digital Speedometer Project
215         </p>
216
217         <p>
218             Python Flask Web Application
219         </p>
220
221     </footer>
```

```
222
223 </div>
224
225
226 <script>
227
228 let rpm = 0;
229
230 let running = false;
231
232 let tripDistance = 0;
233
234 let lastTime = null;
235
236 const wheelDiameter = 0.66;
237
238
239 function calculateSpeed(currentRPM) {
240
241     const circumference =
242         Math.PI * wheelDiameter;
243
244     const metresPerMinute =
245         currentRPM * circumference;
246
247     const speed =
248         (metresPerMinute * 60) / 1000;
249
250     return speed;
251
252 }
253
254
255 function updateDisplay() {
256
257     const speed =
258         calculateSpeed(rpm);
259
260     speedValue.textContent =
261         speed.toFixed(2);
262
263     speedInfo.textContent =
264         speed.toFixed(2);
265
266     rpmValue.textContent =
267         rpm;
268
```

```
    rpmInfo.textContent =
        rpm;


    if (running && rpm > 0) {

        statusValue.textContent =
            "VEHICLE MOVING";

        statusValue.className =
            "status moving";

    } else {

        statusValue.textContent =
            "VEHICLE STOPPED";

        statusValue.className =
            "status stopped";

    }

}


function updateTripDistance() {

    if (!running || rpm <= 0) {

        lastTime = null;

        return;

    }


    const currentTime =
        Date.now();


    if (lastTime === null) {

        lastTime = currentTime;

        return;

    }
```

```
316
317
318     const elapsedSeconds =
319         (currentTime - lastTime) / 1000;
320
321
322     const speed =
323         calculateSpeed(rpm);
324
325
326     const distance =
327         speed * elapsedSeconds / 3600;
328
329
330     tripDistance += distance;
331
332
333     tripDistanceElement.textContent =
334         tripDistance.toFixed(3);
335
336
337     lastTime = currentTime;
338
339 }
340
341
342 document
343     .getElementById("increaseBtn")
344     .addEventListener("click", function() {
345
346         rpm += 50;
347
348         if (rpm > 300) {
349             rpm = 300;
350         }
351
352         updateDisplay();
353
354     });
355
356
357 document
358     .getElementById("decreaseBtn")
359     .addEventListener("click", function() {
360
361         rpm -= 50;
362
```

```
363         if (rpm < 0) {
364             rpm = 0;
365         }
366 
367         updateDisplay();
368 
369     });
370 
371 
372 document
373     .getElementById("startBtn")
374     .addEventListener("click", function() {
375 
376         running = true;
377 
378         lastTime = Date.now();
379 
380         updateDisplay();
381 
382     });
383 
384 
385 document
386     .getElementById("stopBtn")
387     .addEventListener("click", function() {
388 
389         running = false;
390 
391         lastTime = null;
392 
393         rpm = 0;
394 
395         updateDisplay();
396 
397     });
398 
399 
400 document
401     .getElementById("resetBtn")
402     .addEventListener("click", function() {
403 
404         running = false;
405 
406         rpm = 0;
407 
408         tripDistance = 0;
409 
```

```
        lastTime = null;

        tripDistanceElement.textContent =
            "0.000";

        updateDisplay();

    });


setInterval(function() {

    updateTripDistance();

}, 1000);


updateDisplay();

</script>

</body>

</html>
```

Listing 5.2: Main Web Interface – index.html

5.4 CSS Code

The visual appearance of the digital speedometer is controlled by CSS.

```

* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

body {
    font-family: Arial, Helvetica, sans-serif;
    background: #eef2f7;
    min-height: 100vh;
    padding: 30px;
}

.container {
```

```
    max-width: 900px;
    margin: auto;
}

header {
    background: #1f2937;
    color: white;
    text-align: center;
    padding: 30px;
    border-radius: 15px 15px 0 0;
}

header h1 {
    font-size: 32px;
    margin-bottom: 8px;
}

header p {
    font-size: 16px;
}

.speed-card,
.control-card,
.info-card,
.formula-card {
    background: white;
    margin-top: 20px;
    padding: 30px;
    border-radius: 15px;
    box-shadow:
        0 5px 20px rgba(0, 0, 0, 0.08);
}

.speed-card {
    text-align: center;
    padding: 40px 20px;
}

.display-title {
    font-size: 18px;
    font-weight: bold;
    letter-spacing: 2px;
    color: #555;
}

.speed-display {
    margin-top: 15px;
```

```
}

#speedValue {
    font-size: 80px;
    font-weight: bold;
    color: #111827;
}

.unit {
    font-size: 24px;
    font-weight: bold;
    color: #555;
}

.status {
    display: inline-block;
    margin-top: 20px;
    padding: 10px 25px;
    border-radius: 25px;
    font-weight: bold;
}

.status.stopped {
    background: #fee2e2;
    color: #991b1b;
}

.status.moving {
    background: #dcfce7;
    color: #166534;
}

.control-card h2,
.info-card h2,
.formula-card h2 {
    text-align: center;
    margin-bottom: 15px;
}

.description {
    text-align: center;
    color: #666;
    margin-bottom: 20px;
}

.rpm-display {
    text-align: center;
```

```
    font-size: 40px;
    font-weight: bold;
    margin: 20px;
}

.button-row {
    display: flex;
    justify-content: center;
    gap: 15px;
    flex-wrap: wrap;
    margin-top: 15px;
}

button {
    border: none;
    border-radius: 8px;
    padding: 13px 25px;
    font-size: 16px;
    font-weight: bold;
    cursor: pointer;
    transition: 0.2s;
}

button:hover {
    transform: translateY(-2px);
    opacity: 0.9;
}

.control-button {
    background: #374151;
    color: white;
}

.start-button {
    background: #16a34a;
    color: white;
}

.stop-button {
    background: #dc2626;
    color: white;
}

.reset-button {
    background: #2563eb;
    color: white;
}
```

```
157
158 .info-grid {
159     display: grid;
160     grid-template-columns:
161         repeat(2, 1fr);
162     gap: 15px;
163 }
164
165 .info-item {
166     background: #f3f4f6;
167     padding: 20px;
168     border-radius: 10px;
169     text-align: center;
170 }
171
172 .info-label {
173     display: block;
174     color: #666;
175     font-size: 14px;
176     margin-bottom: 8px;
177 }
178
179 .info-value {
180     font-size: 20px;
181     font-weight: bold;
182 }
183
184 .formula-card {
185     text-align: center;
186 }
187
188 .formula-card p {
189     font-size: 20px;
190     font-weight: bold;
191     margin-top: 15px;
192 }
193
194 footer {
195     text-align: center;
196     padding: 25px;
197     color: #666;
198 }
199
200 @media (max-width: 600px) {
201
202     body {
203         padding: 10px;
```

```
204     }
205
206     header h1 {
207         font-size: 24px;
208     }
209
210     #speedValue {
211         font-size: 55px;
212     }
213
214     .info-grid {
215         grid-template-columns: 1fr;
216     }
217
218     button {
219         width: 100%;
220     }
221 }
```

Listing 5.3: CSS Styling – style.css

Chapter 6

Output Snapshots

The following screenshots should be included as evidence of the working of the Digital Speedometer software prototype.

6.1 Initial System State

The initial system state is obtained when the application is opened before increasing the simulated RPM.

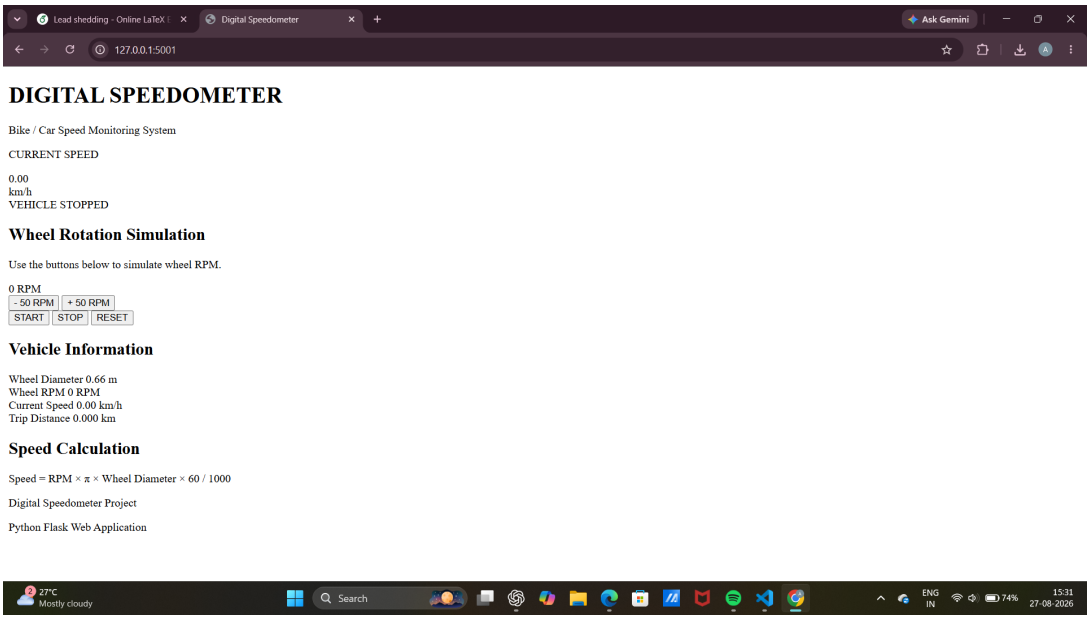


Figure 6.1: Digital Speedometer Initial State

In the initial state, the RPM is zero and the displayed speed is 0.00 km/h.

6.2 Low-Speed Operation

The RPM is increased to 100 RPM using the RPM control button.

For a wheel diameter of 0.66 m, the corresponding calculated speed is approximately 12.44 km/h.

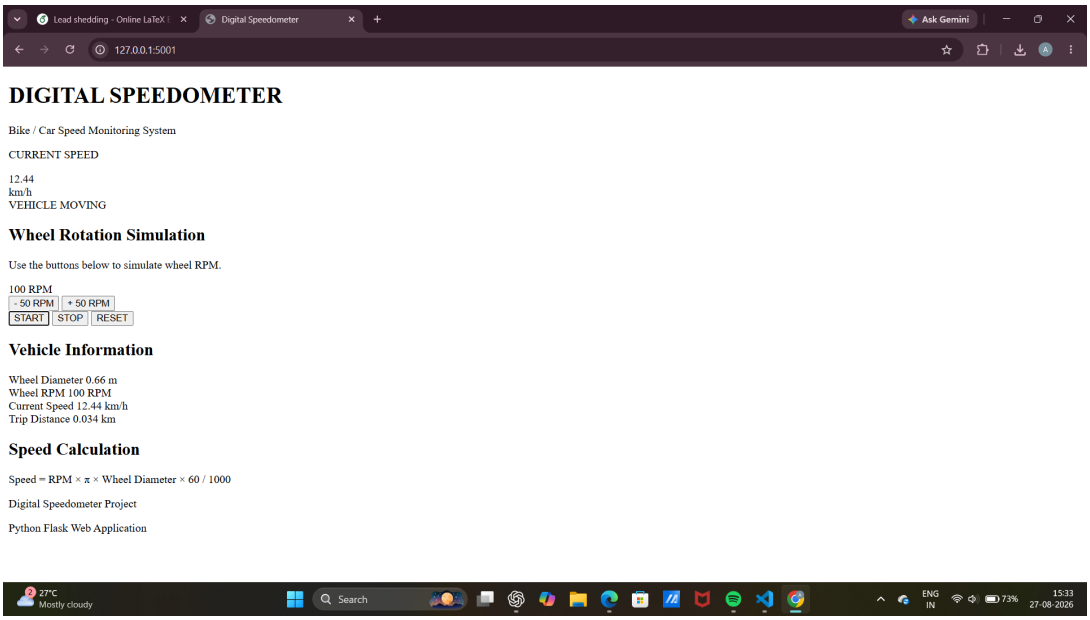


Figure 6.2: Digital Speedometer at 100 RPM

6.3 Medium-Speed Operation

The RPM is increased to 200 RPM.

The corresponding calculated speed is approximately 24.88 km/h.

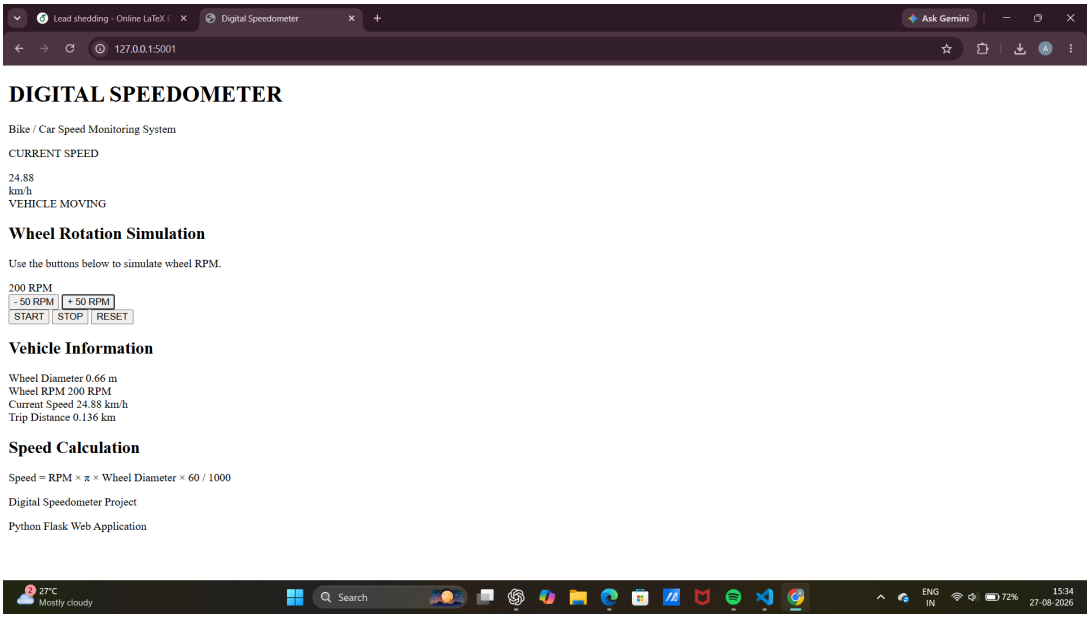


Figure 6.3: Digital Speedometer at 200 RPM

6.4 High-Speed Operation

The RPM is increased to 300 RPM.

The corresponding calculated speed is approximately 37.32 km/h.

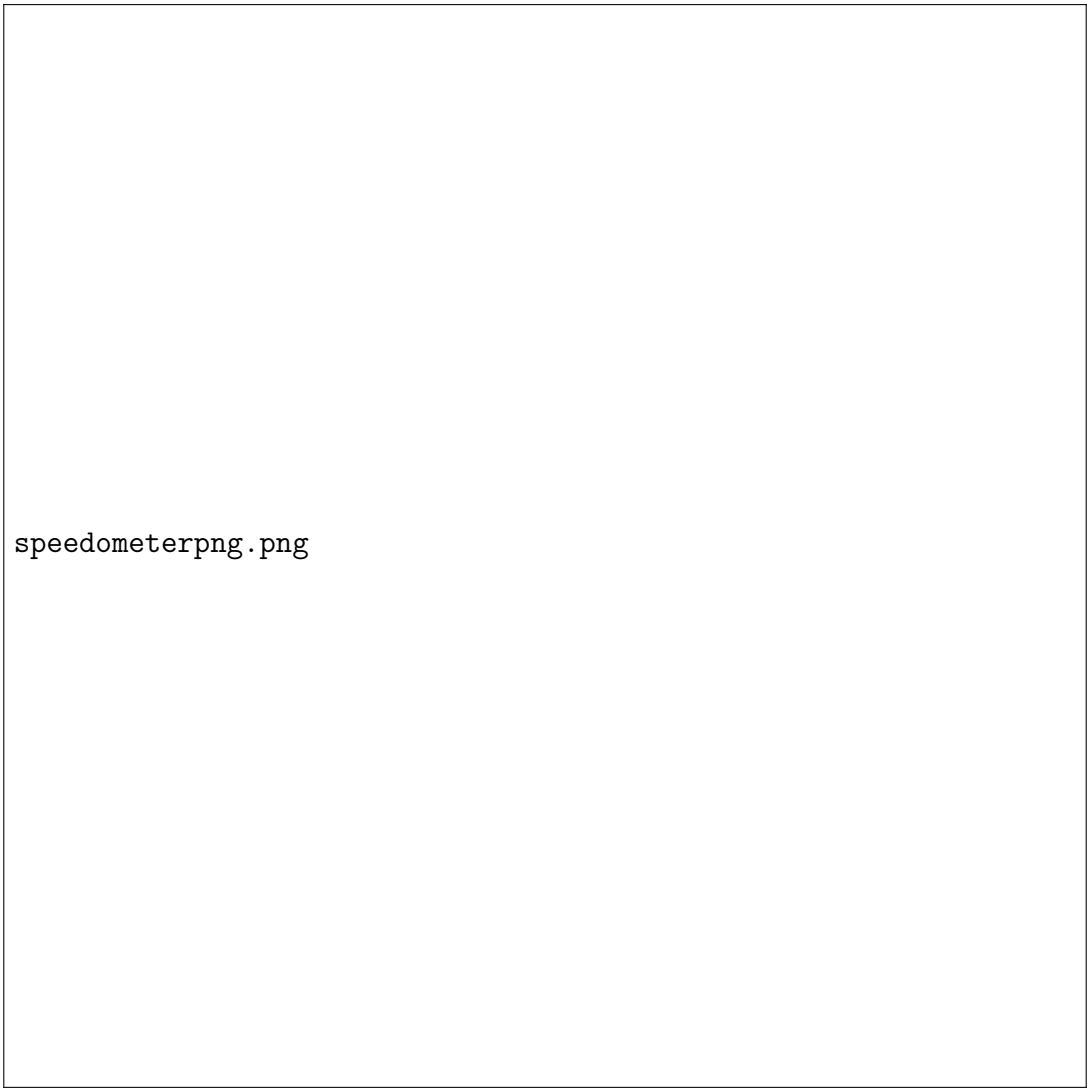


Figure 6.4: Digital Speedometer at 300 RPM

6.5 Flask Server

The Flask application is executed through the VS Code terminal.

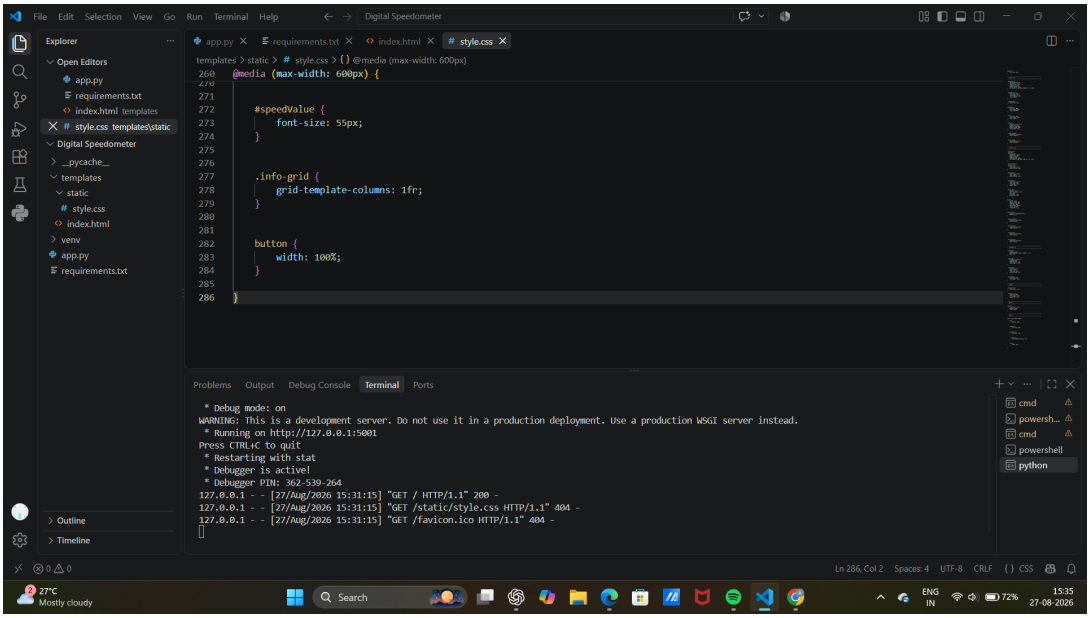


Figure 6.5: Flask Server Running in VS Code

Chapter 7

Result and Analysis

7.1 Result

The Digital Speedometer software prototype was successfully implemented using Python Flask, HTML, CSS, and JavaScript.

The system successfully demonstrates the conversion of simulated wheel RPM into vehicle speed.

The RPM control buttons allow the user to change the simulated wheel rotation. The calculated speed is immediately displayed on the digital dashboard.

The START, STOP, and RESET controls provide basic control over the simulated vehicle operation.

The prototype also displays the trip distance while the simulated vehicle is running.

7.2 Testing

The following test cases were used to verify the speed calculation.

Table 7.1: Speed Calculation Test Results

RPM	Wheel Diameter (m)	Speed (km/h)
0	0.66	0.00
50	0.66	6.22
100	0.66	12.44
150	0.66	18.66
200	0.66	24.88
250	0.66	31.10
300	0.66	37.32

7.3 Analysis

The test results show that the calculated speed increases proportionally with wheel RPM when the wheel diameter remains constant.

For the selected wheel diameter of 0.66 m, increasing RPM from 100 to 200 RPM approximately doubles the calculated speed from 12.44 km/h to 24.88 km/h.

Similarly, at 300 RPM, the calculated speed reaches approximately 37.32 km/h.

This confirms that the mathematical relationship used by the prototype is functioning correctly.

The button-based RPM input also provides a simple method of testing different operating conditions without requiring a physical wheel or sensor.

7.4 Advantages

The developed prototype has the following advantages:

1. Simple and user-friendly interface.
2. Easy RPM control using buttons.
3. Real-time speed calculation.
4. No physical vehicle is required for software testing.
5. Wheel diameter can be used as a calibration parameter.
6. Trip distance can be monitored.

7. Easy to run using Python Flask.
8. Can be extended to a hardware-based implementation.

7.5 Limitations

The current prototype has some limitations:

1. RPM is simulated rather than measured using a physical sensor.
2. The prototype does not directly communicate with a vehicle wheel.
3. The system depends on a computer and web browser for demonstration.
4. The wheel diameter is fixed at 0.66 m in the current implementation.

Chapter 8

Conclusion

The **Digital Speedometer** project was successfully developed as a web-based software prototype using Python Flask, HTML, CSS, and JavaScript.

The system calculates vehicle speed from simulated wheel RPM and wheel diameter. The use of RPM control buttons provides a simple and reliable method for demonstrating different speed conditions.

The calculated speed was tested at different RPM values and the results showed the expected proportional relationship between RPM and vehicle speed.

The project demonstrates the fundamental principle of digital speed measurement and provides a foundation for a future hardware-based speedometer.

In an actual vehicle implementation, the simulated RPM input can be replaced by data obtained from a Hall-effect sensor, IR sensor, wheel encoder, or another suitable sensing mechanism.

Future Scope

The project can be further improved in the following ways:

1. Replace the simulated RPM buttons with a real Hall-effect sensor.
2. Integrate an IR sensor or wheel encoder for wheel rotation measurement.
3. Interface the system with a microcontroller.
4. Add a physical LCD or OLED display.
5. Add GPS-based speed measurement.
6. Provide adjustable wheel diameter calibration.
7. Store trip distance and speed history.
8. Add maximum-speed monitoring.
9. Add warning alerts for excessive speed.
10. Develop a mobile application.

References

1. Project specification, *Digital Speedometer for Bike/Car*.
2. Python Software Foundation, *Python Documentation*.
3. Flask Documentation, *Flask Web Framework Documentation*.
4. HTML, CSS and JavaScript documentation used during implementation.